ESGI · ASSEMBLEUR X86

TP 2 — Chaînes, octets et appels

Guide étudiant — six exercices, le code est fourni

Enseignant : **Youssef YOUNES**

Année universitaire 2026-2027

Polys générés à l'aide d'une IA — contenu relu et validé par l'enseignant.

À RETENIR

Ce TP continue le TP 1 dans les mêmes conditions : un exercice après l'autre, une seule idée nouvelle à chaque fois, le code complet donné dans ce guide et dans le fichier `corrige.asm` de chaque exercice. Lisez-le, comprenez chaque ligne, recopiez-le, vérifiez avec `make test`, puis regardez-le tourner avec `make trace` ou `make debug`. Vos fonctions n'ont toujours pas d'arguments : elles travaillent sur des variables partagées avec `main.c`.

Le cœur du TP, ce sont les exercices 00 à 04. L'exercice 05 est un bonus pour ceux qui ont terminé. Les encarts « Au débogueur » ne sont pas obligatoires : faites d'abord passer `make test`, puis revenez-y si le temps le permet.

1. Démarrer

Mettez le dépôt à jour, puis allez dans le premier exercice :

```
git pull
cd exercices/tp2-chaines/ex00-octets
make check
```

Les commandes sont celles du TP 1. Rappel :

Commande	Effet
`make test`	Compile et vérifie votre programme sur plusieurs entrées. Il faut que tout soit `PASS`.
`make run`	Compile et exécute. Vous tapez l'entrée au clavier.
`make trace`	Exécute instruction par instruction et affiche les registres. `HEX=1` pour les voir en hexadécimal, ce qui est utile dès qu'on manipule des adresses ou des octets.
`make debug BREAK=nom`	Ouvre `gdb` arrêté à l'entrée de la fonction `nom`. La fiche débogueur distribuée avec ce guide rappelle les commandes. Dans ce TP, les fichiers déclarent la fonction en premier : `make debug` tout court s'arrête donc sur la bonne fonction.

2. Un caractère, c'est un octet

Au TP 1, toutes nos valeurs étaient des entiers de 4 octets (`dd` , `eax` , `add esi, 4`). Un texte, c'est différent : chaque caractère occupe un seul octet, et cet octet contient un nombre entre 0 et 255, le code ASCII du caractère.

Caractère	`'A'`	`'Z'`	`'a'`	`'z'`	`'0'` … `'9'`
Code	65	90	97	122	48 … 57

Trois conséquences que les exercices vont utiliser l'une après l'autre :

- Une minuscule moins 32 donne sa majuscule : `'a'` = 97, `'A'` = 65. Les lettres se suivent dans l'ordre alphabétique.
- La fin d'un texte est marquée par un octet qui vaut 0. Le mot `bonjour` occupe 8 octets : les 7 lettres puis un 0. C'est ainsi que `printf` en C sait où s'arrêter, et c'est ainsi que nos boucles sauront s'arrêter.
- Avancer d'un caractère, c'est avancer d'un octet : `inc esi` , et non `add esi, 4` .

En mémoire, le mot `bonjour` rangé dans la variable `texte` ressemble à ceci :

```
adresse    texte   texte+1  texte+2  texte+3  texte+4  texte+5  texte+6  texte+7
contenu     'b'      'o'      'n'      'j'      'o'      'u'      'r'       0
code         98      111      110      106      111      117      114       0
```

Pour manipuler un octet, trois écritures nouvelles :

Écriture	Signification
`texte: times 101 db 0`	Réserve 101 octets à 0 (`db` = define byte, un octet ; `times 101` = répété 101 fois). De quoi ranger 100 caractères et le 0 final.
`movzx eax, byte [texte]`	eax = l'octet rangé à l'adresse `texte` . `byte` précise qu'on lit un seul octet ; `movzx` (move with zero extend) met les trois autres octets de `eax` à zéro, pour que `eax` contienne exactement le code, entre 0 et 255.
`cmp byte [esi], 0`	Compare l'octet pointé par `esi` avec 0. Sans `byte` , l'assembleur ne saurait pas s'il doit comparer 1, 2 ou 4 octets.
`sub byte [esi], 32`	Retire 32 à l'octet pointé par `esi` , en mémoire. C'est ainsi qu'on modifie un texte sur place.
`mov dl, [c]`	`dl` est l'octet le plus bas du registre `edx` . Un registre de 8 bits, à la taille d'un caractère.

> **À RETENIR**
> Au débogueur, la commande qui rend tout cela visible est `x/8cb texte` : huit octets à partir de `texte` , affichés à la fois en nombre et en caractère. Et `x/s texte` affiche le texte entier comme une chaîne.

3. Exercice 00 — Octets

Dossier `exercices/tp2-chaines/ex00-octets` . Le `main.c` lit un mot (sans espace, 100 caractères au plus) et le range dans la variable `texte` de votre fichier. Vous écrivez deux fonctions d'une seule ligne :

```
section .text
global code_premier, code_troisieme
```

```
section .data
global texte
texte:  times 101 db 0

section .text

code_premier:                       ; eax = code du 1er caractère
    movzx eax, byte [texte]         ; l'octet à l'adresse texte, étendu à 32 bits
    ret

code_troisieme:                     ; eax = code du 3e caractère
    movzx eax, byte [texte+2]       ; 2 octets plus loin (un caractère = 1 octet)
    ret
```

`make test` doit afficher 4 `PASS`. Avec `make run` et le mot `bonjour` :

```
premier   : 'b' = 98
troisieme : 'n' = 110
```

Au débogueur. Avant même d'exécuter la première instruction, regardez le texte en mémoire :

```
make debug
(gdb) x/s texte
0x810b048:  "bonjour"
(gdb) x/8cb texte
0x810b048:  98 'b'  111 'o' 110 'n' 106 'j' 111 'o' 117 'u' 114 'r' 0 '\000'
(gdb) x/8xb texte
0x810b048:  0x62  0x6f  0x6e  0x6a  0x6f  0x75  0x72  0x00
(gdb) si
16      ret
(gdb) p $eax
$1 = 98
(gdb) p/c $eax
$2 = 98 'b'
```

Les trois `x` montrent la même mémoire : comme chaîne, comme octets en caractères, comme octets en hexadécimal. Le huitième octet est le 0 final. `p/c` affiche un nombre comme caractère.

4. Exercice 01 — Longueur

Dossier `ex01-longueur`. C'est la boucle du TP 1 (exercice 04), avec deux différences : on ne connaît pas le nombre de tours à l'avance, on s'arrête quand on rencontre l'octet 0 ; et on avance de 1, pas de 4.

Algorithme	Assembleur
adresse = texte	`mov esi, texte`
longueur = 0	`mov eax, 0`
tant que l'octet à l'adresse ≠ 0 :	`boucle:` puis `cmp byte [esi], 0` puis `je fin`
longueur = longueur + 1	`inc eax`

adresse = adresse + 1	`inc esi`
(recommencer)	`jmp boucle`
retourner longueur	`fin:` puis `ret`

```
longueur:
    mov  esi, texte         ; adresse = premier octet du texte
    mov  eax, 0             ; longueur = 0
boucle:
    cmp  byte [esi], 0      ; l'octet courant est-il la marque de fin ?
    je   fin                ; oui : terminé
    inc  eax                ; longueur = longueur + 1
    inc  esi                ; octet suivant (+1, un caractère = 1 octet)
    jmp  boucle
fin:
    ret                     ; résultat dans eax
```

Avec `make trace HEX=1 IN=tests/02-un-caractere.in` (le mot `a`), on voit `esi` passer de `0x0810b048` à `0x0810b049` : un seul octet de plus, et le deuxième `cmp byte [esi], 0` sort de la boucle.

Au débogueur. `display` avec un format : `display/x $esi` affiche `esi` en hexadécimal après chaque pas.

```
make debug
(gdb) display $eax
(gdb) display/x $esi
(gdb) si
...
(gdb) x/s $esi
0x810b049:  "onjour"
(gdb) p/c *(char*)$esi
$1 = 111 'o'
(gdb) break fin
(gdb) continue
Breakpoint 2, fin () at longueur.asm:24
1: $eax = 7
2: /x $esi = 0x810b04f
(gdb) x/2xb $esi
0x810b04f:  0x00  0x00
```

Après un tour, `x/s $esi` montre le texte à partir de la position courante : la première lettre a déjà été comptée. À la sortie, `esi` pointe sur le 0 final, et `eax` vaut 7.

> **À RETENIR**
> `inc esi` et non `add esi, 4`. Si vous avancez de 4, vous sautez trois lettres sur quatre, et vous pouvez même enjamber le 0 final : la boucle continue alors dans la mémoire qui suit, jusqu'à trouver un 0 par hasard. Résultat faux, ou `Segmentation fault`.

5. Exercice 02 — Compter

Dossier `ex02-compter`. Le `main.c` lit un caractère `c` puis un mot. Vous comptez combien de fois `c` apparaît. Nouveauté : garder un caractère dans un registre. Un caractère tient sur un octet, on utilise donc la partie basse de `edx`, nommée `dl` :

```
edx (32 bits) :  [ ........ 24 bits ........ ][ dl : 8 bits ]
```

```
compter:
    mov  esi, texte          ; adresse = premier octet
    mov  eax, 0              ; compte = 0
    mov  dl, [c]             ; dl = le caractère cherché (8 bits)
boucle:
    cmp  byte [esi], 0       ; fin de la chaîne ?
    je   fin
    cmp  [esi], dl           ; l'octet courant est-il le caractère cherché ?
    jne  suivant             ; non : on passe
    inc  eax                 ; oui : compte = compte + 1
suivant:
    inc  esi                 ; octet suivant
    jmp  boucle
fin:
    ret
```

`cmp [esi], dl` n'a pas besoin de `byte` : `dl` fait 8 bits, l'assembleur en déduit qu'il compare un octet. Avec `make run` et l'entrée `a banane` : `'a' apparait 2 fois dans "banane"`.

Au débogueur. Après les trois premières instructions :

```
make debug
(gdb) si
(gdb) si
(gdb) si
(gdb) p/c $dl
$1 = 97 'a'
(gdb) p/x $edx
$2 = 0x40800861
(gdb) p c
$3 = 97 'a'
(gdb) break fin
(gdb) continue
(gdb) p $eax
$4 = 2
```

`p/x $edx` se termine par `61`, c'est-à-dire 97 en hexadécimal : `dl` est bien le dernier octet de `edx`. Les trois autres octets contiennent ce qu'il y avait avant, `mov dl` n'y touche pas.

6. Exercice 03 — Majuscules

Dossier `ex03-majuscules`. Jusqu'ici nous lisions la mémoire. Ici nous l'écrivons : chaque minuscule est remplacée par sa majuscule, au même endroit. Il faut aussi un encadrement : seuls les octets entre `'a'` et `'z'` changent.

```
majuscules:
    mov   esi, texte         ; adresse = premier octet
boucle:
    movzx eax, byte [esi]    ; eax = octet courant
    cmp   eax, 0             ; fin de la chaîne ?
    je    fin
```

```
    cmp   eax, 'a'          ; plus petit que 'a' (97) ?
    jl    suivant           ; oui : pas une minuscule, on ne touche à rien
    cmp   eax, 'z'          ; plus grand que 'z' (122) ?
    jg    suivant           ; oui : pas une minuscule
    sub   byte [esi], 32    ; minuscule : on écrit octet - 32 au même endroit
suivant:
    inc   esi               ; octet suivant
    jmp   boucle
fin:
    ret
```

Deux comparaisons à la suite, chacune avec son saut vers `suivant` : c'est la traduction de « si l'octet est entre `'a'` et `'z'` ». `'a'` entre apostrophes est accepté par l'assembleur, qui le remplace par 97. La fonction ne retourne rien : `main.c` affiche directement `texte` après l'appel.

Au débogueur. Avec `IN=tests/02-melange.in` (`Esgi-2026`), regardez le texte avant, pendant, après :

```
make debug IN=tests/02-melange.in
(gdb) x/s texte
0x810b048:  "Esgi-2026"
(gdb) si
(gdb) si
(gdb) p/c $eax
$1 = 69 'E'
(gdb) break fin
(gdb) continue
(gdb) x/s texte
0x810b048:  "ESGI-2026"
```

Le premier octet est `'E'` (69) : plus petit que 97, le `jl suivant` saute par-dessus le `sub` . Le `s` (115) passe les deux tests et devient `S` (83). À la fin, la mémoire a changé : c'est `x/s texte` qui le prouve, pas un registre.

> **À RETENIR**
> Le test `04-bornes` (`az@AZ[{`) vérifie les frontières : `a` et `z` doivent changer, `@` (64), `[` (91) et `{` (123) ne doivent pas. Si vous oubliez le `jg suivant` , `{` devient `[` et ce seul test échoue.

7. Exercice 04 — Appeler

Dossier `ex04-appeler` . Jusqu'ici, seul `main.c` appelait vos fonctions. Ici, une de vos fonctions en appelle une autre, deux fois, avec `call` .

- `call carre` fait deux choses : il note l'adresse de l'instruction suivante (l'adresse de retour) sur la pile, puis saute à `carre` .
- `ret` reprend cette adresse sur la pile et y saute : on revient juste après le `call` .

Entre vos fonctions, les valeurs passent par les registres : `carre` attend son nombre dans `eax` et y laisse le résultat.

```
carre:                      ; eax = eax * eax
    imul eax, eax
    ret                     ; retour juste après le call

somme_carres:               ; eax = a*a + b*b
```

```
    mov  eax, [a]           ; eax = a
    call carre              ; eax = a*a
    mov  ecx, eax           ; ecx = a*a, mis de côté
    mov  eax, [b]           ; eax = b
    call carre              ; eax = b*b
    add  eax, ecx           ; eax = b*b + a*a
    ret
```

Pourquoi `mov ecx, eax` ? Parce que le second `call carre` va écraser `eax`. `make trace` avec `3 4` montre le va-et-vient :

```
=== somme_carres ===
instruction (exécutée)      eax    ebx    ecx    edx    esi
mov  eax, [a]                 3    ...    ...    ...    ...
call carre                    3    ...    ...    ...    ...
imul eax, eax                 9    ...    ...    ...    ...
ret                           9    ...    ...    ...    ...
mov  ecx, eax                 9    ...      9    ...    ...
mov  eax, [b]                 4    ...      9    ...    ...
call carre                    4    ...      9    ...    ...
imul eax, eax                16    ...      9    ...    ...
ret                          16    ...      9    ...    ...
add  eax, ecx                25    ...      9    ...    ...
ret                          25    ...      9    ...    ...
retour : eax =               25
```

Au débogueur. C'est ici que la pile devient visible. `esp` est l'adresse du sommet de la pile ; `call` y dépose 4 octets, `ret` les reprend.

```
make debug
(gdb) p/x $esp
$1 = 0x40800cbc
(gdb) si
(gdb) si
carre () at appeler.asm:16
16      imul eax, eax
(gdb) p/x $esp
$2 = 0x40800cb8
(gdb) x/wx $esp
0x40800cb8: 0x0804977e
(gdb) x/i *(int*)$esp
   0x804977e <somme_carres+10>: mov    ecx,eax
(gdb) si
(gdb) si
somme_carres () at appeler.asm:23
23      mov  ecx, eax
(gdb) p/x $esp
$3 = 0x40800cbc
```

Après le `call`, `esp` a diminué de 4, et le mot qu'il désigne est l'adresse de `mov ecx, eax` : exactement l'instruction qui suit le `call`. Après le `ret`, `esp` est revenu à sa valeur. Deux façons d'avancer sur un `call` : `si` entre dans `carre`, `ni` (next instruction) exécute l'appel entier et s'arrête juste après.

> **À RETENIR**
> La pile sert d'abord à ça : se souvenir d'où l'on vient. Au TP 3, elle servira aussi à passer les arguments, comme en C.

8. Exercice 05 — Afficher (bonus)

Dossier `ex05-afficher`. Cet exercice est un bonus : faites-le si les cinq premiers passent tous leurs tests. Un programme n'a pas le droit d'écrire lui-même sur l'écran : il le demande au système d'exploitation. Sous Linux 32 bits, la demande se fait avec l'instruction `int 0x80`, une interruption : le processeur suspend votre programme, passe la main à Linux, qui exécute le service demandé et rend la main à l'instruction suivante. Les paramètres de la demande sont dans quatre registres :

Registre	Valeur	Signification
`eax`	4	Le numéro du service : 4 = `write`, écrire des octets.
`ebx`	1	Où écrire : 1 = la sortie standard, l'écran.
`ecx`	`texte`	L'adresse du premier octet à écrire (sans crochets : l'adresse).
`edx`	`[longueur]`	Le nombre d'octets à écrire. `main.c` a rangé la longueur du mot dans cette variable.

```
afficher:
    push ebx                ; on sauvegarde ebx (il appartient à l'appelant)
    mov  eax, 4             ; service n° 4 : write
    mov  ebx, 1             ; 1 = sortie standard (l'écran)
    mov  ecx, texte         ; adresse des octets à écrire
    mov  edx, [longueur]    ; nombre d'octets
    int  0x80               ; appel au système : Linux écrit les octets
    pop  ebx                ; on rend ebx intact
    ret
```

Pourquoi `push ebx` et `pop ebx` ? Par convention, `ebx` appartient à l'appelant : `main.c` a le droit de compter dessus après l'appel. `eax`, `ecx` et `edx`, eux, peuvent être modifiés librement, c'est ce que nous faisons depuis le TP 1. `push` dépose la valeur sur la pile, `pop` la reprend : la même pile que celle du `call`.

Avec `make run` et `bonjour` :

```
bonjour
(7 octet(s) ecrit(s) par l'appel systeme)
```

La première ligne a été écrite par votre `int 0x80`, la seconde par le `printf` de `main.c`.

Au débogueur. Juste avant le `int 0x80`, les quatre registres sont prêts :

```
make debug
(gdb) p/x $ebx
$1 = 0x810aff4
(gdb) si
(gdb) si
(gdb) si
(gdb) si
```

```
(gdb) si
21      int  0x80
(gdb) info registers eax ebx ecx edx
eax            0x4                 4
ebx            0x1                 1
ecx            0x810b048           135311432
edx            0x7                 7
(gdb) x/s $ecx
0x810b048:  "bonjour"
(gdb) si
bonjour
(gdb) p $eax
$2 = 7
```

Le `si` sur `int 0x80` fait apparaître `bonjour` à l'écran : Linux a écrit. Au retour, `eax` contient la réponse du système, le nombre d'octets réellement écrits. Continuez jusqu'au `ret` puis dans `main.c`, et vérifiez `p/x $ebx` : `0x810aff4`, la valeur de départ, grâce au `pop`.

À RETENIR
Sur Mac Apple Silicon, le `si` sur `int 0x80` exécute aussi le `pop ebx` qui suit, et vous arrivez directement sur `ret` : c'est une particularité de l'émulateur, pas de votre programme. Sur PC, vous vous arrêtez sur `pop ebx`.

9. Erreurs fréquentes

Message ou symptôme	Cause et solution
`operation size not specified`	`cmp [esi], 0` ou `sub [esi], 32` sans taille : écrivez `byte [esi]`. (Avec un registre comme `dl`, la taille est connue.)
`mismatch in operand sizes`	`mov eax, byte [texte]` : un registre de 4 octets et une lecture de 1 octet. Utilisez `movzx eax, byte [texte]`, ou `mov al, [texte]`.
Longueur fausse, souvent trop grande	`add esi, 4` au lieu de `inc esi` : vous enjambez des octets, parfois le 0 final.
`FAIL ... délai dépassé`	Boucle infinie : `cmp esi, 0` (l'adresse) au lieu de `cmp byte [esi], 0` (l'octet), ou `inc esi` oublié.
Seul le test `04-bornes` échoue (ex 03)	Il manque une des deux comparaisons de l'encadrement (`jl` ou `jg`).
Résultat `a + b*b` (ex 04)	`mov ecx, eax` placé avant le premier `call` : vous avez mis `a` de côté, pas `a*a`.
`Segmentation fault` après `call`	Il manque le `ret` à la fin de `carre` : le processeur continue dans ce qui suit en mémoire.

Rien ne s'affiche (ex 05)	`mov ecx, [texte]` avec crochets : `ecx` contient les 4 premières lettres au lieu de l'adresse. Ou `edx` à 0.
`make debug` ne s'arrête pas	Vous avez donné un `BREAK=` mal orthographié. Dans ce TP, `make debug` tout court suffit.

10. Et après

Les `corrige.asm` se terminent chacun par un exercice supplémentaire, avec son code : `nb_chiffres`, `position`, `minuscules`, `cube`, le retour à la ligne par appel système. Au TP 3, vos fonctions recevront enfin leurs arguments par la pile, comme en C : le `call` de l'exercice 04 et le `push` / `pop` de l'exercice 05 sont exactement les deux mécanismes qu'il utilise. C'est le dossier `exercices/tp3-fonctions`.